

Homework 1 — Tokenization and Word Vectors

Natural Language Processing (2026) · Dept. of Applied Artificial Intelligence, SeoulTech

Student ID	22101355	Name	Gyuwan Choi
------------	----------	------	-------------

Three to six sentences per question. None of these asks for an essay. Every answer must cite a number from your own tables or point at a line of your own output; paste what a question refers to above your answer, so the reader can check it.

1. Results and observations

Part 1. Byte Pair Encoding

Table 1. Lines, characters and words of the three slices (results/00_corpus.csv)

slice	lines	characters	words
train	175,152	106,424,973	19,999,973
valid	4,217	2,644,579	499,763
small	8,126	4,989,679	938,931

Output: The first 50 merge rules printed by cell 1-3

0 t + h → th	10 e + n → en	20 i + t → it	30 en + t → ent	40 u + r → ur
1 i + n → in	11 a + l → al	21 t + o → to	31 a + d → ad	41 d + e → de
2 th + e → the	12 s + t → st	22 o + u → ou	32 s + e → se	42 v + e → ve
3 a + n → an	13 a + r → ar	23 i + c → ic	33 a + m → am	43 o + l → ol
4 e + r → er	14 o + f → of	24 h + e → he	34 o + m → om	44 T + he → The
5 o + n → on	15 an + d → and	25 i + on → ion	35 b + e → be	45 i + r → ir
6 e + d → ed	16 a + s → as	26 l + e → le	36 a + c → ac	46 i + m → im
7 r + e → re	17 in + g → ing	27 r + o → ro	37 l + y → ly	47 - + @ → -@
8 a + t → at	18 e + s → es	28 c + h → ch	38 w + as → was	48 @ + -@ → @-@
9 o + r → or	19 i + s → is	29 i + l → il	39 f + or → for	49 o + w → ow

1-3 (5 points) What do the earliest merges have in **common**, and from roughly which rank do morpheme-like pieces begin to appear? Quote two or three merge rules by their rank as evidence.

Answer: The earliest BPE merges capture **the most frequent character pairs** in English and rapidly form common function words: rank 0 merges t + h → th, rank 2 th + e → the, rank 14 o + f → of and rank 15 an + d → and. These merges reflect raw frequency rather than linguistic structure, which also explains why ranks 47–48 build the WikiText hyphen marker @-@.

Morpheme-like suffixes begin to appear from roughly rank 6 onward, well within the first 50 merges: rank 6 e + d → ed, rank 17 in + g → ing, rank 25 i + on → ion and rank 37 l + y → ly. Because these suffixes occur across many different stems, their pair counts accumulate over many words (the sum in the rule below), so they are merged earlier than pieces tied to a single less frequent stem.

$$(a^*, b^*) = \arg \max_{(a, b)} \sum_w f(w) \cdot n_w(a, b)$$

Selection rule in bpe.py: $f(w)$ is the frequency of word w and $n_w(a, b)$ the count of the adjacent pair in its current split. Ties go to the alphabetically smaller pair; training stops when the maximum is below 2.

Table 2. Your implementation against tokenizers (results/15_impl_time.csv)

implementation	merges	vocabulary	train_sec
mine (pure python)	1000	1,612	1.44
tokenizers (rust)	-	1,612	0.31

Output: The vocabulary Jaccard similarity, the token counts over 200 lines, and the two segmentations of the probe sentence, from cell 1-4

	mine	library
vocabulary size	1,612	1,612
tokens on 200 validation lines	49,796	49,558
tokens incl. <sp> between words	76,136	–
vocabulary Jaccard similarity, as is	0.215	
vocabulary Jaccard similarity, Ġ stripped	0.482	

word	mine (23)	library (25)
The	The	ĠThe
unbelievable	un be li ev able	Ġun b el ie v able
tokenization	to k en iz ation	Ġto k en iz ation
of	of	Ġof
2026	20 2 6	Ġ20 2 6
costs	co st s	Ġc ost s
\$3.50.	\$ 3 . 50 .	Ġ\$ 3 . 5 0 .

1-4 (5 points) Your vocabulary and the library's overlap only partly, and the two produce a different number of tokens on the same text. Name at least three causes and say which of them the printed evidence supports: pre-tokenization unit, byte-level handling, the treatment of the space before a word, tie breaking, special tokens.

Answer: At the same vocabulary size (1,612) the two vocabularies overlap little, and stripping the space marker Ġ more than doubles the Jaccard similarity (0.215 → 0.482), so the treatment of the space before a word is a major contributor. Byte-level handling gives the two a different base alphabet: my implementation starts from the 610 characters seen in the corpus (1,612 – 2 specials – 1,000 merges), while the library starts from 256 byte symbols and so has 1,356 merges, and it represents a non-ASCII character as a sequence of bytes rather than as one symbol. The two also learned different merges: on the probe mine keeps 50 as one token while the library splits it into 5 | 0, a difference inside a run of digits that reflects the learned merge list rather than pre-tokenization alone. Special tokens change the count: on 200 lines mine yields 49,796 subwords but 76,136 ids once <sp> is counted, against 49,558 for the library. The pre-tokenization unit and tie breaking may also contribute, but this output does not isolate them.

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, J_{\text{as is}} = 0.215 \rightarrow J_{\text{Ġ stripped}} = 0.482$$

Part 2. Word vectors

Table 3. Vocabulary size against validation token count (results/16_tokenizers.csv)

tokenizer	vocabulary	valid_tokens	train_sec
bpe-2k	2,000	869,819	2.2
bpe-8k	8,000	658,749	3.1
bpe-32k	32,000	557,986	3.7
word-level	259,319	499,763	0.0

Output: The raw-vocabulary coverage of the four models, from cell 2-4

* candidates = 50,123 / eval words = 1,319

model	Word2Vec vocabulary	raw coverage
word-level	85,403	0.9530
bpe-2k	1,922	0.0121
bpe-8k	7,880	0.0288
bpe-32k	31,122	0.0895

Table 4. Coverage and scores after word vectors are built (results/25_word_level_eval.csv)

model	eval_kind	vectors	coverage	analogy_acc	wordsim_spearman
word-level	word (restricted)	50,061	0.953	0.1227	0.3164
bpe-2k	subword mean	50,123	1.000	0.1891	0.1701
bpe-8k	subword mean	50,123	1.000	0.1419	0.2622
bpe-32k	subword mean	50,123	1.000	0.1287	0.3057

2-4 (6 points) Coverage of the evaluation words is about 0.78 for the word-level model and near zero for the BPE models. Explain the gap, state whether the low BPE figure means those models cannot represent the evaluation words, and say what the BPE models still do not solve once the out-of-vocabulary problem is gone.

Answer: In my run the raw coverage is 0.953 for the word-level model (higher than the template's 0.78) but only 0.0121, 0.0288 and 0.0895 for bpe-2k, 8k and 32k. Raw coverage counts exact keys only, and a BPE model stores a word-initial piece with the space marker Ġ or splits the word into several pieces, so a bare evaluation word is rarely a key; in the nearest-neighbour probe of 2-4, none of the five words (city, king, water, music, government) is in the raw vocabulary of bpe-2k. The low figure does not mean the BPE models cannot represent these words: built as the mean of their subword vectors (below), all three reach a coverage of 1.0 in Table 4. What remains unsolved is quality. The mean of the pieces does not capture words whose meaning is not the sum of their parts, so bpe-2k scores 0.0054 on semantic analogies against 0.1693 for the word-level model (Table 5).

$$v(w) = \frac{1}{|S(w)|} \sum_{s \in S(w)} v(s)$$

Word vector of a BPE model: $S(w)$ is the subword segmentation of “ w ” with its leading space (Ġ).

Output: The per-section accuracy of each model, from cell 2-5b

section	tests	word-level	bpe-2k	bpe-8k	bpe-32k
capital-common-countries	meaning	0.332 (506)	0.004 (506)	0.168 (506)	0.391 (506)
capital-world	meaning	0.152 (2,595)	0.003 (4,524)	0.017 (4,524)	0.087 (4,524)
currency	meaning	0.000 (130)	0.000 (866)	0.000 (866)	0.002 (866)
city-in-state	meaning	0.156 (2,403)	0.001 (2,467)	0.065 (2,467)	0.142 (2,467)
family	meaning	0.213 (380)	0.061 (506)	0.047 (506)	0.190 (506)
gram1-adjective-to-adverb	form	0.017 (992)	0.530 (992)	0.382 (992)	0.143 (992)
gram2-opposite	form	0.041 (756)	0.244 (812)	0.100 (812)	0.057 (812)
gram3-comparative	form	0.026 (1,332)	0.175 (1,332)	0.090 (1,332)	0.049 (1,332)
gram4-superlative	form	0.020 (1,056)	0.436 (1,122)	0.165 (1,122)	0.078 (1,122)
gram5-present-participle	form	0.006 (992)	0.663 (1,056)	0.410 (1,056)	0.146 (1,056)
gram6-nationality-adjective	form	0.482 (1,445)	0.395 (1,599)	0.360 (1,599)	0.451 (1,599)
gram7-past-tense	form	0.071 (1,560)	0.112 (1,560)	0.138 (1,560)	0.081 (1,560)
gram8-plural	form	0.009 (1,190)	0.375 (1,332)	0.233 (1,332)	0.071 (1,332)
gram9-plural-verbs	form	0.044 (812)	0.226 (870)	0.149 (870)	0.043 (870)

Table 5. Semantic accuracy, syntactic accuracy, questions scored (results/25b_sections.csv)

model	semantic_acc	syntactic_acc	semantic_n	syntactic_n	scored
word-level	0.1693	0.0951	6,014	10,135	16,149
bpe-2k	0.0054	0.3417	8,869	10,675	19,544
bpe-8k	0.0389	0.2275	8,869	10,675	19,544
bpe-32k	0.1175	0.1380	8,869	10,675	19,544

Table 6. The combined table (results/26_summary.csv)

model	eval_kind	vectors	valid_tokens	raw_cov	coverage	analogy	wordsim	train_sec
word-level	word (restricted)	50,061	499,763	0.9530	0.953	0.1227	0.3164	32.8
bpe-2k	subword mean	50,123	869,819	0.0121	1.000	0.1891	0.1701	75.3
bpe-8k	subword mean	50,123	658,749	0.0288	1.000	0.1419	0.2622	61.9
bpe-32k	subword mean	50,123	557,986	0.0895	1.000	0.1287	0.3057	88.9

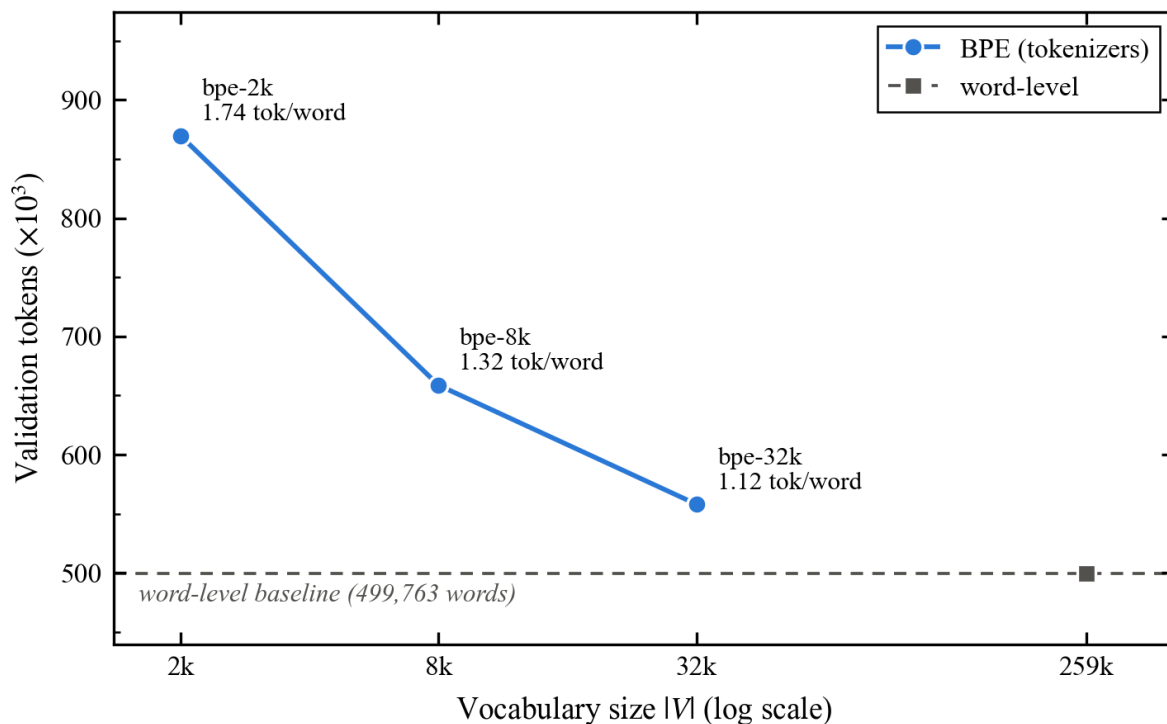


Figure 1. Vocabulary size against validation token count (results/26_vocab_vs_tokens.png)

2-6 (6 points) As the vocabulary grows from 2,000 to 32,000, what improves and what degrades? Address the sequence length, the training time and the evaluation scores together, citing one number for each.

Answer: From 2k to 32k the validation sequence becomes shorter, from 869,819 to 557,986 tokens, so each Word2Vec epoch has fewer tokens to process; the measured training time, however, does not fall with it (75.3 s, 61.9 s and 88.9 s for 2k, 8k and 32k), so this run gives no clear evidence that a larger vocabulary trains faster, and wall-clock time on a multi-threaded run also varies between runs. Word similarity improves: the WordSim-353 correlation rises from 0.170 to 0.306 and semantic analogy accuracy from 0.0054 to 0.1175. Overall analogy accuracy, however, falls from 0.189 to 0.129, because syntactic accuracy drops from 0.3417 to 0.1380. A larger vocabulary keeps more words whole, which helps meaning but loses the shared suffix pieces that made form analogies easy.

$$\frac{869,819 - 557,986}{869,819} \approx 35.9\%$$

Reduction in validation tokens from bpe-2k to bpe-32k

Table 7. The three settings you swept (results/27_sweep.csv)

model	window	analogy_acc	wordsim_spearman	train_sec
word-level	2	0.0832	0.2503	27.3
word-level	5	0.1276	0.3255	34.2
word-level	10	0.1520	0.3876	40.1

2-7 (4 points) State the trend your sweep shows and give the setting you would choose. If the three values differ by less than the run-to-run variation, say so rather than reading a trend into them.

Answer: Widening the window from 2 to 5 to 10 raises analogy accuracy (0.0832 → 0.1276 → 0.1520) and the WordSim-353 correlation (0.2503 → 0.3255 → 0.3876), at the cost of longer training (27.3 s → 34.2 s → 40.1 s). To estimate run-to-run variation I compare two runs of the same setting, window 5: the sweep gives 0.1276 / 0.3255 and the model from 2-2 gives 0.1227 / 0.3164, a difference of 0.005 in accuracy and 0.009 in correlation. Each step of the sweep (0.044 and 0.024 in accuracy, 0.075 and 0.062 in correlation) is several times larger than these differences, although more repetitions are needed to confirm the trend. I would choose window 10, best on both metrics for about 17 % more training time than window 5; larger windows were not tested.

$$\Delta_{\text{run}} = |0.1276 - 0.1227| \approx 0.005 \ll \Delta_{\text{window}} \approx 0.03$$

Same setting (window 5) trained twice, against the step between adjacent window values (analogy accuracy)

Part 3. Discussion

No new experiments. Answer from the tables above.

3-1 (5 points) Does the model with the highest overall analogy accuracy represent word meaning best? Answer from the per-section accuracy and the number of scored questions in the 2-5b tables above.

Answer: No. bpe-2k has the highest overall analogy accuracy (0.1891), but its semantic accuracy is the lowest of the four (0.0054); its score comes from form sections such as gram5-present-participle (0.663) and gram1-adjective-to-adverb (0.530), where shared suffix pieces make the answer easy. The word-level model has the best semantic accuracy (0.1693, e.g. capital-world 0.152 against 0.003 for bpe-2k) and the best WordSim-353 correlation (0.3164). The totals are also not directly comparable: the word-level model scored only 16,149 questions because a question with an unknown word is skipped, while every BPE model scored 19,544.

3-2 (4 points) Going from one-hot vectors through the distributional hypothesis to dense vectors, what is given up and what is gained? One paragraph.

Answer: One-hot vectors are exact and trivially interpretable, one dimension per word, but they encode no similarity at all: in 2-1 all 1,000 random pairs of distinct words have cosine 0.0 in a 20,000-dimensional space. The distributional hypothesis replaces a word's identity with its contexts, and dense vectors compress those contexts into 100 dimensions, which gains similarity and generalisation; the word-level model places throne, Egbert, Northumbria, pope, emperor next to king. What is given up is the meaning of individual dimensions and the guarantee that distinct words stay orthogonal, and the vectors now depend on training data: a word seen fewer than 5 times (min_count) gets no vector at all. Static dense vectors also still do not distinguish different senses of the same word, a limitation they share with one-hot vectors.

$$\cos(e_i, e_j) = \frac{e_i \cdot e_j}{\|e_i\| \|e_j\|} = 0, \text{ for } i \neq j$$

Two distinct one-hot vectors share no non-zero dimension, so their dot product is always zero.

2. Self-check output

Paste the complete output of python test_bpe_public.py below.

```
Bash
$ python test_bpe_public.py
PASS test_count_words
PASS test_ids_are_in_range
PASS test_merge_order
PASS test_round_trip
PASS test_tokenize_matches_slide21
PASS test_vocab_growth

0 failed
```